

Perceptrón Simple y Multicapa

Detección de fraude y clasificación de dígitos

Britez, Santiago Kantor, Marcos Sánchez Toledo, Mariano
Sistemas de Inteligencia Artificial — TP3

Instituto Tecnológico de Buenos Aires

2026



Contexto y método

Ejercicio 1: Detección de fraude

Ejercicio 2: Clasificación de dígitos

Ejercicio 3: Superar el 98 %

Conclusiones

Qué hicimos y cómo lo medimos

Tres experimentos

- ▶ **Ej.1 – Fraude:** perceptrón simple lineal vs. sigmoideo. TinyModel como aproximación de BigModel.
- ▶ **Ej.2 – Dígitos:** MLP con grid search de lr, arquitectura y optimizador.
- ▶ **Ej.3 – Más datos:** superar el 98 % con aumento de datos; ponderación y sobremuestreo descartados.

Protocolo experimental

- ▶ **Fraude:** división 70/15/15; selección en validación y evaluación en test. MSE, MAE y F1.
- ▶ **Dígitos:** comparación de arquitecturas, tasas y optimizadores; selección por accuracy de validación y evaluación en test.
- ▶ **Más datos:** selección por accuracy de validación; en test, accuracy, F1 macro y recall por dígito.

Ej.1 – Objetivo y diseño experimental

Problema

CompanyX dispone de un **BigModel** que estima la probabilidad de fraude, pero su inferencia es costosa. El objetivo es construir un **TinyModel** mucho más pequeño que aproxime esa salida.

Datos

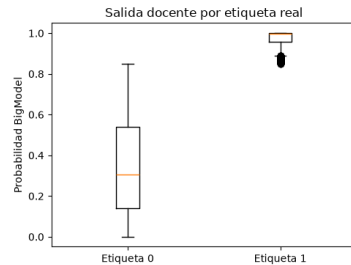
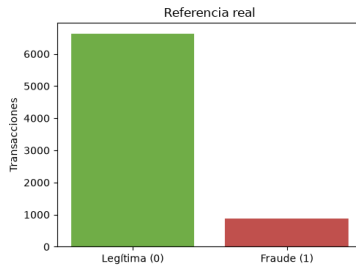
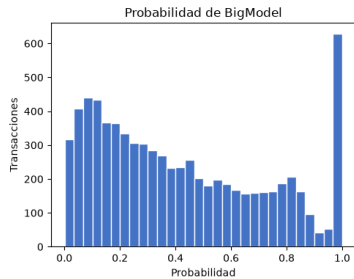
- ▶ 7.500 transacciones.
- ▶ 11 variables numéricas.
- ▶ 869 fraudes: **11,59 %**.
- ▶ Sin valores nulos ni duplicados.
- ▶ 9 variables de entrada.

Objetivos separados

- ▶ **Entrenamiento:** aproximar p_{BigModel} .
- ▶ **Evaluación:** utilizar `flagged_fraud` como referencia real.
- ▶ La etiqueta real **no se utiliza como objetivo de entrenamiento**.

Pregunta experimental

¿Puede un único perceptrón con salida sigmoidea reproducir de forma útil la salida probabilística del BigModel y, posteriormente, detectar fraude?



Ej.1 – Qué muestran los datos

Composición

- ▶ 7.500 observaciones.
- ▶ 869 fraudes.
- ▶ Clase positiva minoritaria: **11,59 %**.

Salida del BigModel

La distribución de p_{BigModel} presenta una separación marcada entre las transacciones legítimas y fraudulentas.

Implicación experimental

La clasificación final no debe evaluarse únicamente con accuracy.

Se analizarán:

- ▶ Precision.
- ▶ Recall.
- ▶ F1.
- ▶ Matriz de confusión.

**Primero aprendemos la salida del BigModel.
Después evaluamos la detección de fraude.**

Ej.1 – ¿Qué función de salida conviene?

Ajuste sobre todas las muestras

Modelo	MSE	MAE	Rango
Lineal crudo	0.02606	0.12945	$[-0,418, 2,067]$
Lineal recortado	0.02266	0.11966	$[0, 1]$
Sigmoide	0.01187	0.08328	$[0,005, 1]$

Nota: Baseline constante: $MSE = 0,09148$.

Resultado y Decisión

La sigmoide obtiene $MSE = 0,01187$ frente al modelo lineal (0,02606), manteniendo además de forma natural la salida en el intervalo $[0, 1]$.

Decisión: Se utiliza el perceptrón **sigmoide** para estudiar la generalización.

Ej.1 – Lineal vs. sigmoide: interpretación

Aspecto	Lineal	Sigmoide
Salida	No acotada	$[0, 1]$
Aprox. probabilidad	Limitada	Adecuada
MSE	0.02606	0.01187
MAE	0.12945	0.08328
Rango predicho	$[-0,42, 2,07]$	$[0,005, 1]$
Elección	—	Sí

Underfitting

No severo. La sigmoide reduce el MSE un 87 % frente al baseline (0.09148) y un 48 % frente al lineal recortado.

Selección: perceptrón sigmoide → menor MSE, salida naturalmente en $[0, 1]$, mayor potencial de ajuste.

Saturación de capacidades

Un perceptrón simple está limitado por construcción a una única combinación lineal seguida de una activación. **No modela interacciones no lineales entre features.**

Que la sigmoide mejore **no implica que el problema esté resuelto**: solo que una salida acotada y no lineal aproxima mejor al BigModel.

Ej.1 – Métricas de evaluación

Para aproximar al BigModel

$$\text{MSE} = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$$
$$\text{MAE} = \frac{1}{N} \sum_i |y_i - \hat{y}_i|$$

- ▶ MSE penaliza más los errores grandes.
- ▶ MAE es interpretable en la misma escala.
- ▶ Miden la calidad de la **destilación**.

Para detectar fraude

- ▶ **Precision**: alertas que son fraude real.
- ▶ **Recall**: fraudes detectados.
- ▶ **F1**: balance Precision/Recall.
- ▶ **Matriz de confusión**: FP y FN explícitos.
- ▶ **Balanced accuracy**: acierto por clase.

Por qué no accuracy sola

La clase positiva representa solo el **11,6 %**. Un clasificador trivial que nunca marca fraude obtendría accuracy 0,884 pero recall = 0 y F1 = 0. Por eso reportamos métricas sensibles a la clase minoritaria.

Ej.1 – Estrategia de generalización

Split estratificado por `flagged_fraud`

Partición	Muestras	Fraudes
Entrenamiento (70 %)	5.250	608
Validación (15 %)	1.125	130
Test (15 %)	1.125	131

Estandarización ajustada **solo** con entrenamiento y aplicada sin recalcular a validación y test.

Flujo

Train → ajustar pesos

Validación → elegir lr , época y umbral

Train+Val → reentrenar con la configuración fija

Test → evaluar **una sola vez**

Principio

El test **no participa** de ninguna decisión de diseño: se mantiene aislado hasta que la configuración y el umbral están congelados.

Ej.1 – Generalización: selección de hiperparámetros

Protocolo

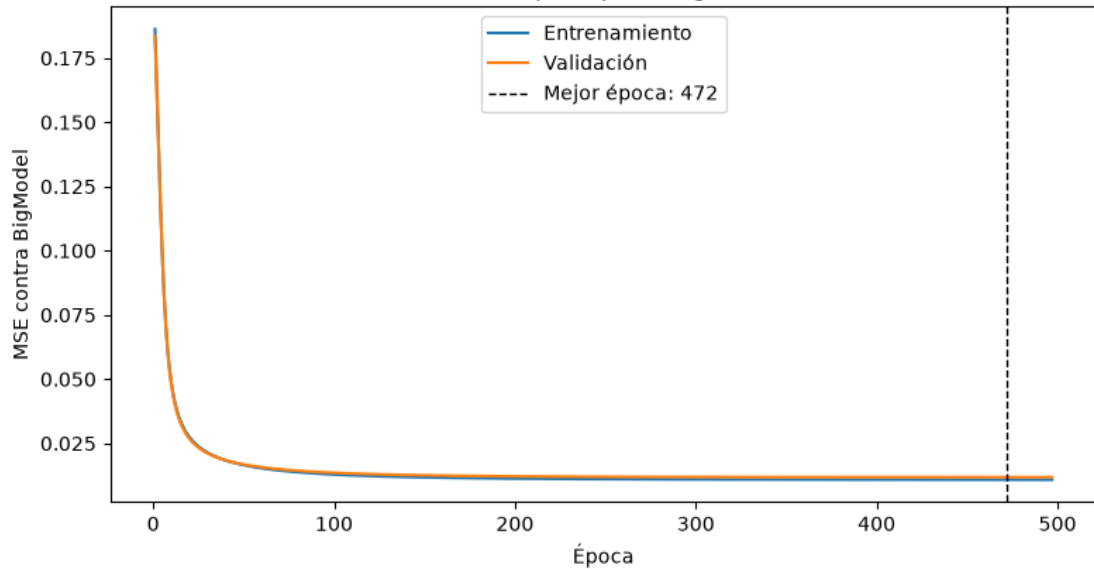
- ▶ Split estratificado: **70 % / 15 % / 15 %**.
- ▶ Estandarización ajustada únicamente sobre entrenamiento.
- ▶ Arquitectura fija: perceptrón con activación sigmoide.
- ▶ Minibatch: 256 | Máximo: 800 épocas | Early stopping: 25 épocas.

Learning rate	Mejor época	MSE validación	MAE validación
0.003	800	0.01432	0.09396
0.010	800	0.01193	0.08151
0.030	472	0.01181	0.08002

Criterio de selección

La configuración se selecciona exclusivamente por el error en **validación**. El conjunto de test permanece reservado para la evaluación final.

Generalización del perceptrón sigmoide ($lr=0.03$)



Configuración seleccionada

$$lr = 0,03$$

$$\text{época} = 472$$

Criterio: menor MSE de validación.

Qué observamos

- ▶ La tasa 0,03 alcanza el menor MSE de validación.
- ▶ La validación mejora hasta una época intermedia.
- ▶ Se evita seleccionar el modelo utilizando el test.

**La siguiente decisión ya no es arquitectónica:
¿qué probabilidad consideramos “fraude”?**

Ej.1 – Selección del umbral

Problema

El TinyModel produce una probabilidad continua. Para tomar una decisión binaria necesitamos definir un umbral:

$$\hat{y} = \begin{cases} 1 & \text{si } p_{\text{Tiny}} \geq t \\ 0 & \text{si } p_{\text{Tiny}} < t \end{cases}$$

Criterio

No se conocen los costos económicos relativos de:

- ▶ falsos positivos (FP),
- ▶ falsos negativos (FN).

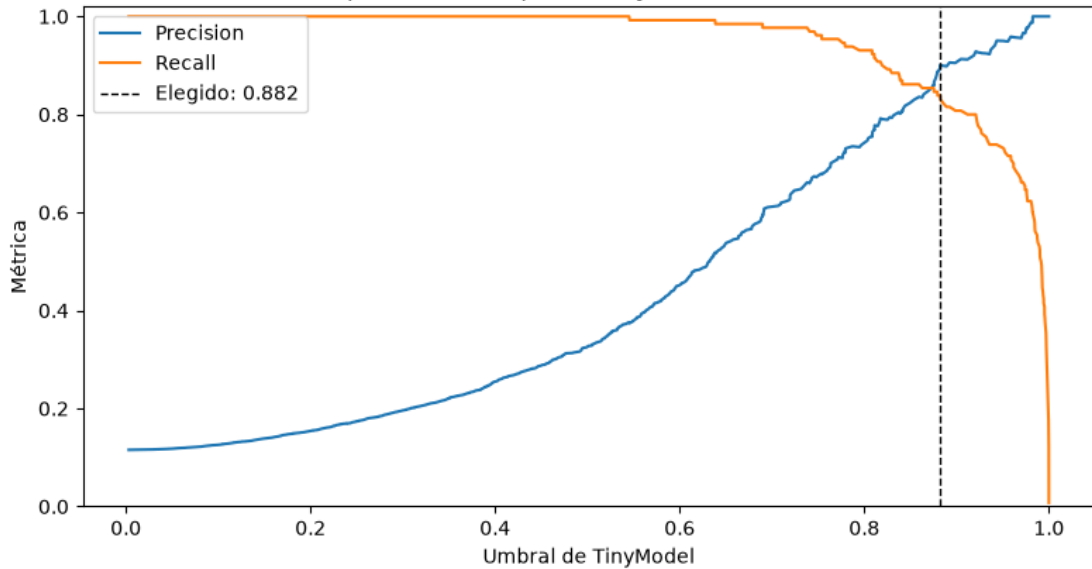
Por eso se selecciona en validación el umbral que maximiza F1.

Resultado en validación

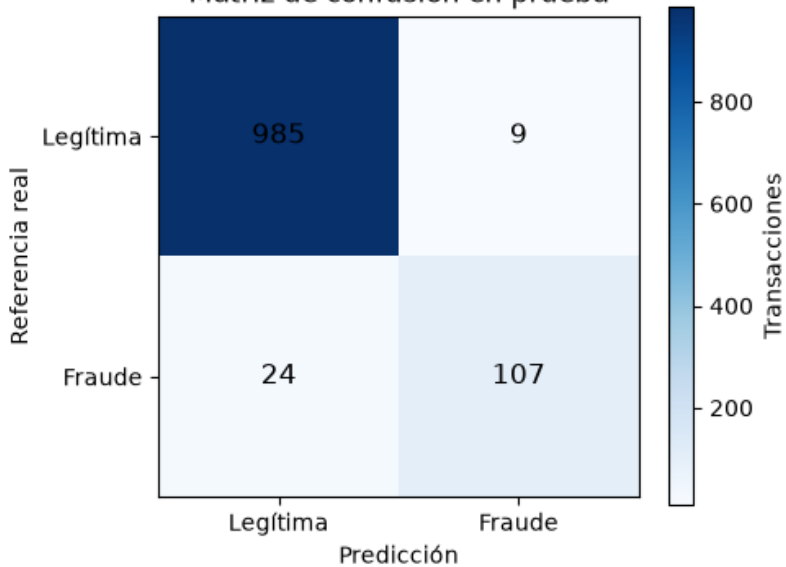
$$t = 0,8822$$

Métrica	Valor
Precision	0.9008
Recall	0.8385
F1	0.8685

Compromiso entre precision y recall en validación



Matriz de confusión en prueba



Ej.1 – Resultado final en test

TinyModel vs. BigModel

Métrica	Test
MSE	0.01020
MAE	0.07398

Error considerablemente menor que el baseline de regresión (MSE = 0,09148).

Detección de fraude

Métrica	Test
Umbral	0.8822
Accuracy	0.9707
Balanced accuracy	0.9040
Precision	0.9224
Recall	0.8168
F1	0.8664
FP / FN	9 / 24

Lectura del resultado

Sobre 1.125 transacciones: **107 de 131 fraudes detectados**, 9 falsos positivos, 24 fraudes sin detectar.

Costo esperado con FP= 1, FN= 20: $9 + 24 \cdot 20 = 489$.

Ej.1 – Modelo propuesto a CompanyX

Ficha ejecutiva

Componente	Valor recomendado
Modelo	Perceptrón simple con activación sigmoide
Entradas	9 features de la transacción (estandarizadas)
Learning rate	0,03
Época seleccionada	472
Umbral de decisión	$t = 0,8822$
MSE test (vs. Big)	0,01020
MAE test (vs. Big)	0,07398
Accuracy	0,9707
Balanced accuracy	0,9040
Precision	0,9224
Recall	0,8168
F1	0,8664
FP / FN	9 / 24

Recomendación

TinyModel aproxima la salida del BigModel con un error bajo y permite tomar decisiones de fraude con $F1 = 0,87$. El umbral debe revisarse si CompanyX estima un costo relativo distinto entre FP y FN.

Resultados principales

1. La activación **sigmoide** reduce el error de aproximación de la salida del BigModel (87 % frente al baseline).
2. La mejor configuración de validación fue $lr = 0,03$ y **472 épocas**.
3. El umbral se fijó **antes de mirar el test**: $t = 0,8822$ (máximo F1 en validación).
4. En test se obtiene: **Balanced acc = 0.904, Precision = 0.922, Recall = 0.817, F1 = 0.866**.

Conclusión experimental

Un perceptrón sigmoide aproxima la salida del BigModel y permite detectar fraude con un umbral fijado en validación.

El umbral no es universal: con costos de negocio (FP= 1, FN= 20), el umbral de máximo F1 implica un costo de 489 sobre test. Si CompanyX prioriza detectar fraude, conviene bajar el umbral.

Objetivo → datos → experimento → decisión

Qué muestra

- ▶ Los 9 pesos y el sesgo a lo largo de las épocas.
- ▶ Comparación directa entre perceptrón lineal y sigmoide.
- ▶ Permite ver la velocidad de convergencia y la estabilidad.

Ver GIF

▶ [Abrir evolución de pesos](#)

(Se abre en el navegador. Requiere conexión.)

Qué muestra

- ▶ Predicción de TinyModel contra p_{BigModel} en cada época.
- ▶ La recta diagonal representa el ajuste perfecto.
- ▶ Se observa cómo la sigmoide se adapta mejor a la forma de la salida docente.

Ver GIF

▶ [Abrir ajuste lineal vs. sigmoide](#)

(Se abre en el navegador. Requiere conexión.)

Tarea

Clasificar imágenes de **28×28 píxeles** como uno de los dígitos del 0 al 9 con un perceptrón multicapa.

Conjuntos provistos

Archivo	Imágenes
digits.csv	12.449
digits_test.csv	2.497

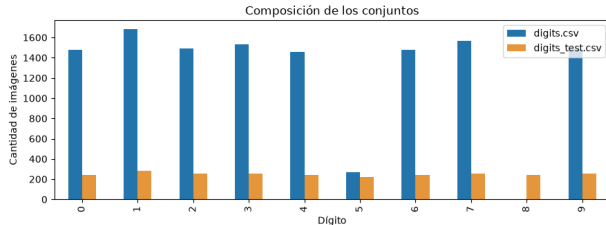
Los píxeles ya están en $[0, 1]$.

Distribución de clases

digits.csv no contiene imágenes etiquetadas como **8** y tiene solo **271** imágenes del 5.

El test sí incluye **243** ochos. Esta diferencia limita el resultado final.

Ej.2 – Composición de los conjuntos



Desequilibrio visible

- ▶ Las clases 0–7 y 9 tienen $\approx 1,500$ imágenes cada una.
- ▶ El **dígito 5** tiene solo **271** (18 % del promedio).
- ▶ El **dígito 8** tiene **0** en train pero **243** en test.

Consecuencia

El dataset de desarrollo **no cubre las 10 clases**. Cualquier métrica que promedie sobre todas las clases va a castigar al modelo por algo que nunca pudo aprender.

Ej.2 – Un ejemplo de cada clase



Observaciones

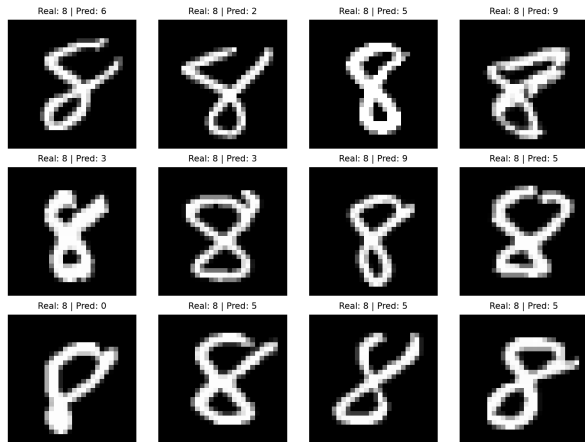
- ▶ Imágenes de 28×28 píxeles, ya normalizadas a $[0, 1]$.
- ▶ Variabilidad alta dentro de cada clase (trazo, inclinación, grosor).
- ▶ La celda del **8** está vacía: no hay muestras en `digits.csv`.

Anticipo

Esa ausencia es la causa directa del **recall = 0** que veremos en test.

Ej.2 – ¿A qué se parecen los 8 que el modelo nunca vio?

Ochos mal clasificados (243 de 243)



Predicciones (243 ochos)

Pred	Cant.	Pred	Cant.
3	80	6	18
5	58	0	12
9	34	1	10
2	19	4	10
		7	2

Lectura geométrica

El 8 se confunde con **3, 5 y 9** (71 %):
trazos curvos cerrados.

Ej.2 – ¿Cómo evaluamos el desempeño?

Métricas

- ▶ **Accuracy:** proporción total de aciertos.
- ▶ **Recall por clase:** fracción detectada de cada dígito.
- ▶ **F1 macro:** promedio del F1 de las diez clases.
- ▶ **Matriz de confusión:** identifica los dígitos confundidos.

Uso de cada conjunto

- ▶ **Train:** ajustar los pesos.
- ▶ **Validación:** seleccionar hiperparámetros por accuracy.
- ▶ **Test:** medir el modelo final una vez fijada la configuración.

Por qué mirar cada clase

La accuracy global puede ocultar el fallo sobre un dígito ausente del entrenamiento. Por eso también mostramos recall y F1 por clase.

Ej.2 – ¿Qué variantes probamos?

Búsqueda de hiperparámetros

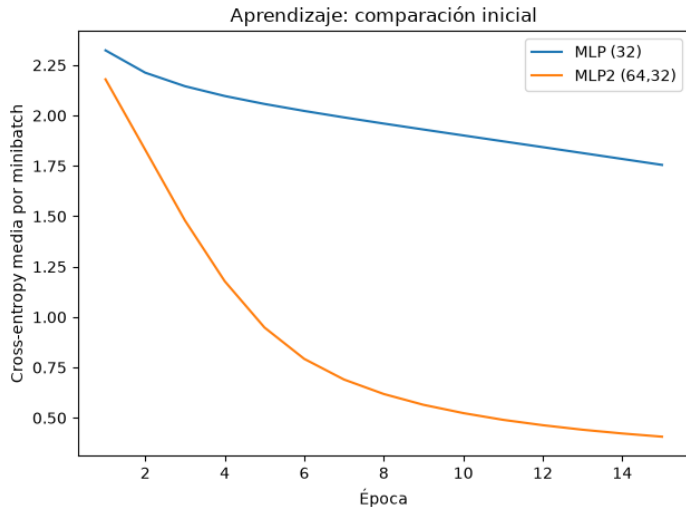
Factor	Valores
Capas ocultas	(32, 16), (64, 32), (128, 64)
Learning rate	0,1, 0,01, 0,001
Optimizador	SGD, momentum, RMSProp, Adam

36 configuraciones, 15 épocas cada una.

Protocolo del notebook

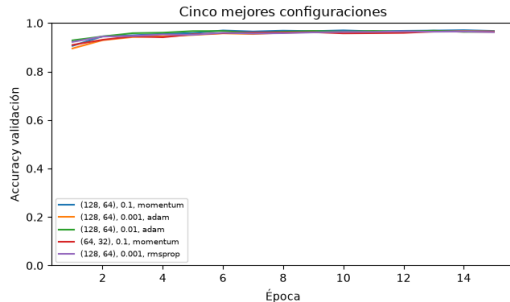
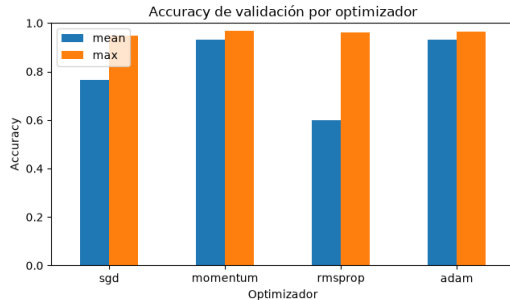
- ▶ `digits.csv`: 9.959 para train y 2.490 para validación.
- ▶ Split estratificado y semilla 42.
- ▶ Minibatches de 256 para ejecutar la grilla.
- ▶ Selección por accuracy de validación.

Ej.2 – Comparación inicial: una capa vs. dos capas



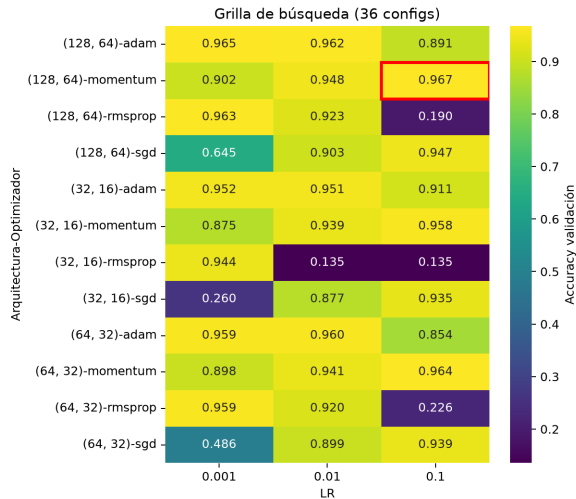
Una red con más capacidad aprende mucho más rápido: MLP(32) termina en loss $\approx 1,75$;
MLP(64,32) en $\approx 0,41$.

Ej.2 – Comparación visual de configuraciones



Accuracy de validación por optimizador y evolución de las cinco mejores configuraciones.

Ej.2 – Mapa completo de la grilla



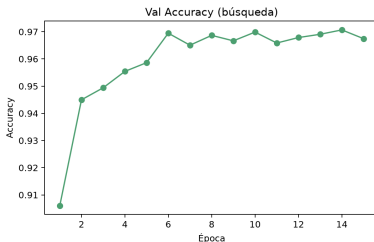
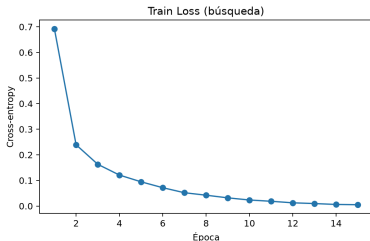
Lectura del heatmap

- ▶ Cada celda es una de las **36 configuraciones**.
- ▶ **Recuadro rojo**: mejor celda ((128, 64), momentum, $lr = 0,1$) con 0,9675.
- ▶ Peor celda: 0,1353 (RMSProp con lr alto).
- ▶ **Rango** de accuracy val: 0,83.

Conclusión

El optimizador domina más que la arquitectura: la misma red varía más de 70 puntos según cómo se optimiza.

Ej.2 – Curvas del mejor modelo (búsqueda, 15 épocas)



Dinámica observada

Métrica	Época 1	Época 15
Train loss	0,6918	0,0051
Val accuracy	0,9060	0,9675

El máximo de validación se alcanza en la **época 14** (0,9707). No hay señales de sobreajuste dentro de las 15 épocas.

Por qué importa

La búsqueda se detiene a las 15 épocas; el reentrenamiento final usa 50. Ese cambio de régimen explica parte de la brecha val→test.

Ej.2 – Selección en validación

Mejores configuraciones tras 15 épocas

Capas	lr	Optimizador	Accuracy val.
(128, 64)	0,1	momentum	0,9675
(128, 64)	0,001	Adam	0,9651
(128, 64)	0,01	Adam	0,9643
(128, 64)	0,001	RMSProp	0,9643
(64, 32)	0,1	momentum	0,9639

Criterio de selección

Se compararon **36 configuraciones** con la misma partición. El ranking usa accuracy de validación; el test quedó reservado.

Modelo elegido

Capas (128, 64), **lr** = 0,1, **momentum**.
Obtuvo 0,9675 de accuracy en validación.

Modelo reentrenado

La configuración elegida se entrenó desde cero con las **12.449 imágenes** de `digits.csv` durante 50 épocas.

Métricas finales

Métrica	Test
Accuracy	0,8662
F1 macro	0,8198
Recall del 8	0,0000

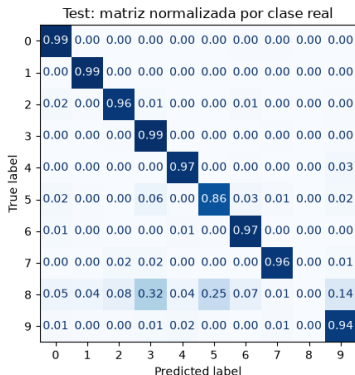
La selección usó 15 épocas; el reentrenamiento final usó 50. Este cambio de duración también puede afectar la diferencia entre validación y test.

Lectura de la brecha

El modelo acertó **0 de 243** ochos en test: no había ejemplos de esa clase en entrenamiento.

En las otras nueve clases acertó **2.163 de 2.254** imágenes (**95,96 %**). Al incluir los ochos, la accuracy total es 86,62 %.

Ej.2 – Errores y conclusión



Matriz de confusión en test, normalizada por dígito real.

Respuesta al enunciado

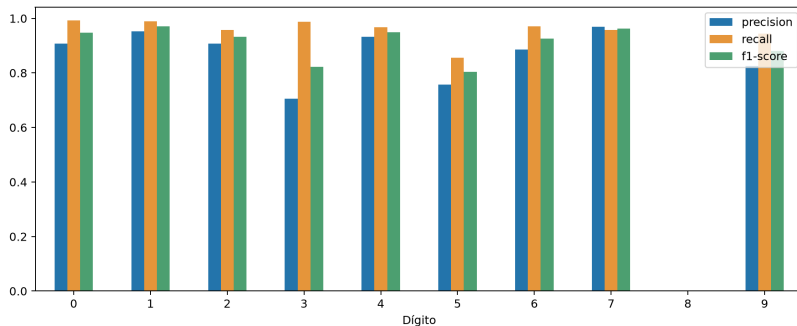
Evaluamos con accuracy, F1 macro y matriz de confusión.

Exploramos arquitectura, tasa de aprendizaje y optimizador.

Límite principal

El dataset inicial no cubre las diez clases. El resultado del dígito 8 motiva estudiar los datos adicionales del ejercicio 3.

Ej.2 – Precision, Recall y F1 por dígito



El outlier

El **dígito 8** tiene precision, recall y F1 en **cero**: nunca apareció en entrenamiento.

Las otras nueve clases tienen F1 entre 0,80 y 0,97.

Macro-promedios

Métrica	Valor
Precision macro	0,7844
Recall macro	0,8623
F1 macro	0,8198

Ej.2 – La brecha train→test, cuantificada

Resumen del modelo final

Métrica	Valor
Accuracy train	1,0000
Accuracy test	0,8662
Balanced accuracy	0,8623
F1 macro test	0,8198
Accuracy sin el 8	0,9596

El modelo acierta **96 %** en las 9 clases conocidas.

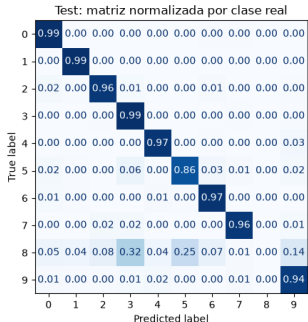
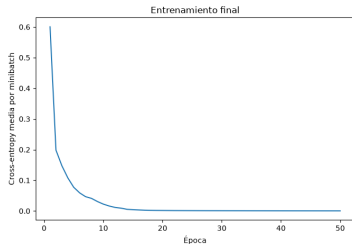
Descomposición de la brecha

- ▶ Brecha train→test: +13,4 pts.
- ▶ Aporte de la clase 8: +9,7 pts.
- ▶ 72,8 % de la brecha total se explica por el dígito ausente.
- ▶ El 27,2 % restante es la brecha real de generalización.

Conclusión

El bajo desempeño en test **no es un fallo del MLP**: es la consecuencia directa de que `digits.csv` no contenga ochos.

Ej.2 – Entrenamiento final del modelo elegido



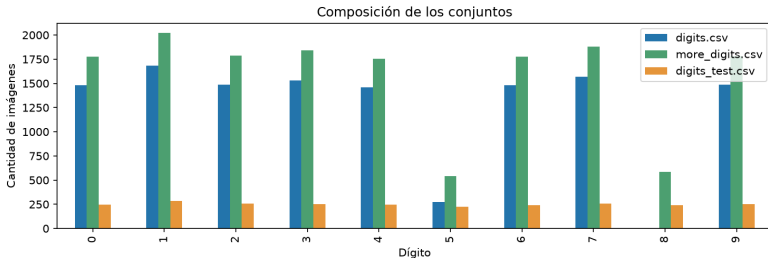
Lectura de la curva

- ▶ Reentrenamiento con las **12.449 imágenes** de `digits.csv`.
- ▶ **50 épocas**, batch de 256, semilla 42.
- ▶ El loss cae de $\sim 0,60$ a $\sim 0,002$.
- ▶ Convergencia estable, sin oscilaciones.

Ojo con la interpretación

Un loss casi nulo en train **no** implica buen test: el modelo memoriza las 9 clases que ve y falla en la 10ª que no ve.

Ej.3 – Objetivo y datos nuevos



Objetivo

- ▶ Accuracy $\geq 98\%$ con un perceptrón multicapa.
- ▶ `digits.csv` + `more_digits.csv`: parámetros e hiperparámetros.
- ▶ `digits_test.csv`: solo evaluación final.

Qué aportan los datos nuevos

- ▶ 15.741 imágenes; **3.689** ya estaban en `digits.csv` y se eliminan.
- ▶ Conjunto de desarrollo: **24.501** imágenes únicas.
- ▶ Aparecen **585** ochos (antes 0); el 5 pasa de 271 a 785.
- ▶ Sin etiquetas contradictorias ni imágenes de test repetidas.

Ej.3 – El efecto de los datos, sin tocar el modelo

Mismo modelo del Ej.2: capas (128, 64), ReLU, momentum, tasa 0,1, batch 256, 50 épocas

Entrenamiento	Imágenes	Accuracy test	Recall del 5	Recall del 8
Ej.2: solo digits.csv	12.449	0,8662	0,857	0,000
+ more_digits.csv	24.501	0,9696	0,937	0,914

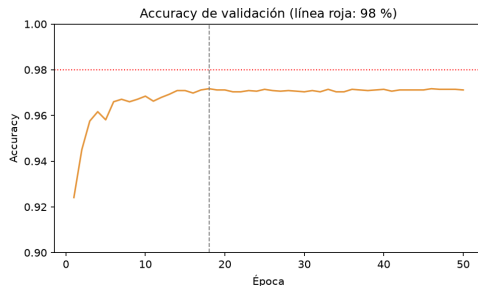
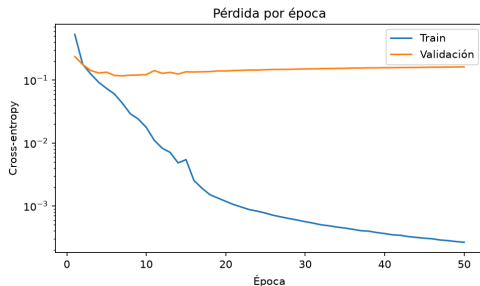
+10,3 puntos solo por los datos

No cambió ninguna técnica ni hiperparámetro.
La mejora viene de que ahora **existen ejemplos del 8** y hay casi el triple de 5.

Pero no alcanza

96,96 % sigue por debajo del 98 %: faltan unos 27 aciertos sobre 2.497. Hace falta mejorar el modelo, no solo sumar datos.

Ej.3 – Diagnóstico: el modelo memoriza



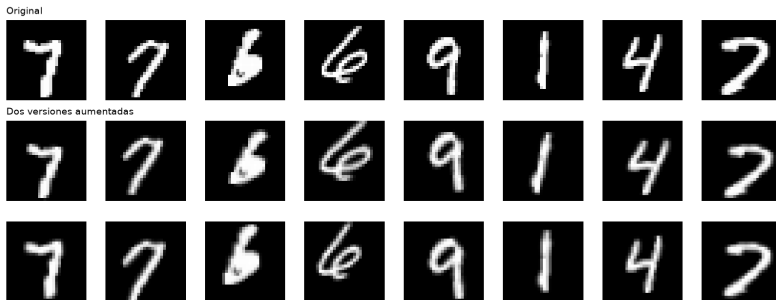
Validación para decidir sin mirar el test

- ▶ Partición estratificada 85/15, semilla 42: 20.825 train y 3.676 validación.
- ▶ Se conservan los pesos de la **mejor época en validación**.

Sobreajuste

- ▶ Train llega a 100 % y su pérdida cae a ~ 0 .
- ▶ Validación se estanca en 97,17 % y su pérdida vuelve a subir.
- ▶ Falta **generalización**, no capacidad.

Ej.3 – Técnica 1: aumento de datos



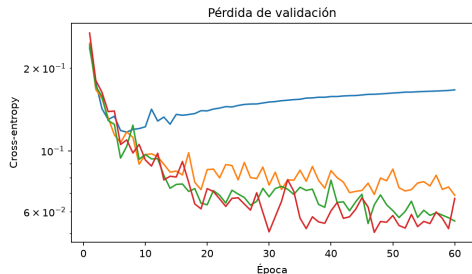
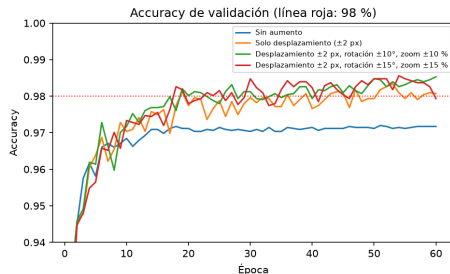
Por qué

Un MLP no tiene noción de posición: un dígito corrido dos píxeles es una entrada distinta. Si la red ve variantes, deja de memorizar imágenes puntuales.

Cómo

- ▶ En cada minibatch: desplazamiento, rotación y zoom aleatorios (interpolación bilineal).
- ▶ La etiqueta no cambia.
- ▶ **Solo en entrenamiento:** validación y test usan las imágenes originales.

Ej.3 – Aumento de datos: elección en validación



Misma red, 60 épocas

Variante	Acc. val. *	Pérdida val.
Sin aumento	0,9717	0,167
Desplazamiento ± 2 px	0,9804	0,069
± 2 px , $\pm 10^\circ$, zoom ± 10 %	0,9844	0,056
± 2 px, $\pm 15^\circ$, zoom ± 15 %	0,9827	0,067

* media de las últimas 5 épocas.

Lectura

- ▶ +1,3 pts en validación.
- ▶ La pérdida de validación ya no sube.
- ▶ Rotación y zoom suman sobre el desplazamiento solo.

Ej.3 – Técnicas 2 y 3: balancear el 5 y el 8

Ponderación de la pérdida por clase

$w_c = (N/(10 n_c))^{\alpha}$: un error en una clase escasa cuesta más.

α	Acc. val.
0 (sin pesos)	0,9844
0,5	0,9824
1	0,9814

Sobremuestreo

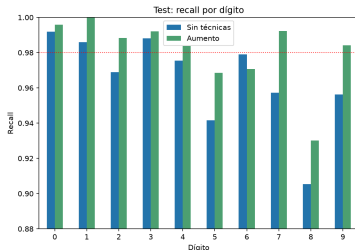
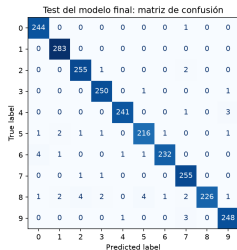
Copias de las clases escasas; cada copia recibe un aumento distinto.

Nivel	Acc. val.
Sin sobremuestreo	0,9844
Parcial (50 %)	0,9829
Completo (100 %)	0,9833

Ninguna mejora: se descartan

Ambas se probaron sobre el modelo con aumento y solo **mueven errores** entre clases (sube el recall de una, baja su precisión). El límite no es el peso del 5 y el 8 sino que hay **pocos ejemplos distintos** (667 y 497 en train). Las diferencias son de 0,1 a 0,3 puntos, con una sola semilla.

Ej.3 – Resultado final en test



Métricas

Métrica	Valor
Accuracy validación	0,9894
Accuracy test	0,9812
F1 macro test	0,9808
Errores en test	47 de 2.497
Recall del 5	0,969
Recall del 8	0,930

Modelo final (elegido en validación)

Capas (128,64), ReLU, momentum, tasa 0,1, batch 256, 100 épocas con aumento (± 2 px, $\pm 10^\circ$, zoom $\pm 10\%$); pesos de la época 96.

Objetivo alcanzado, con poco margen

El 98 % admite 49 errores: el margen es de **2 imágenes**. El 8 concentra 17 de los 47 errores.

Ej.3 – Respuestas al enunciado

(a) ¿Cuál es el mejor resultado?

Accuracy 98,12 % y F1 macro 0,981 en `digits_test.csv`, con el mismo MLP (128, 64) del Ej.2 entrenado con aumento de datos.

(b) ¿Qué técnicas se usaron?

Técnica	Efecto	¿Se adopta?
Aumento de datos	test 96,56 % → 98,12 % (+1,56 pts)	Sí
Ponderación de la pérdida	validación 98,44 % → 98,24 %	No
Sobremuestreo	validación 98,44 % → 98,33 %	No

(c) ¿Qué otros factores influyeron?

Los **datos**: con el modelo sin cambios, sumar `more_digits.csv` lleva el test de 86,62 % a 96,96 %, sobre todo porque aparece el 8.

Límites

Una sola semilla y una sola partición. Entre ejecuciones una misma configuración varía $\sim 0,1$ pts, comparable al margen sobre el 98 %.

Ej.1 – Fraude

- ▶ **Sigmoide** mejor que lineal: salida en $[0, 1]$ y menos de la mitad del MSE.
- ▶ Test: **F1** 0,87 con umbral 0,88 fijado en validación.

Ej.2 – Dígitos

- ▶ Optimizador y tasa pesan más que la arquitectura.
- ▶ Test: 86,6 %, porque `digits.csv` no tiene ningún 8.

Ej.3 – Superar el 98 %

- ▶ Datos nuevos: 86,6 % $\rightarrow$ 97,0 %.
- ▶ Aumento de datos: 98,12 %. Objetivo cumplido por 2 imágenes.

Lo que se repite en los tres ejercicios

- ▶ **Los datos pesan más que el modelo:** las mayores mejoras vinieron de cubrir la clase faltante y de aumentar los datos, no de cambiar la arquitectura.
- ▶ **El test se usa una sola vez:** tasa, época, umbral y técnicas se eligieron en validación.
- ▶ **La métrica depende del problema:** F1 en fraude (clases desbalanceadas); accuracy y recall por dígito en clasificación.

Dar margen al 98 % (Ej.3)

- ▶ Repetir con **varias semillas** y particiones para reportar media y desvío: hoy el margen es menor que el ruido entre ejecuciones.
- ▶ **Ensamble** de varias redes y reentrenar con train + validación.
- ▶ Entrenar más épocas: la mejor fue la 96 de 100.
- ▶ **Regularización** (dropout, L2) y redes más anchas junto con el aumento de datos.
- ▶ Conseguir más ejemplos del 5 y del 8: es el límite que el balanceo no pudo resolver.

Entender mejor el clasificador de dígitos

- ▶ **Robustez al ruido**: evaluar el test con ruido gaussiano de distinta intensidad.
- ▶ **Interpretabilidad**: métodos de atribución para ver qué píxeles deciden cada dígito.

Fraude (Ej.1)

- ▶ Analizar y ajustar la **calibración** de las probabilidades.
- ▶ Fijar el umbral con los **costos reales** de FP y FN de CompanyX.
- ▶ Probar ReLU como activación y construir nuevos features.

¡Gracias!



Preguntas y comentarios.